

COMPLETE_VIDEO_SCRIPTS

Phase 3 Complete Video Scripts

This document contains all 12 video scripts for the Phase 3 video course.

Video 3: Session Management Fundamentals (10 min)

Topics Covered

- Session lifecycle and states
- The 6 session modes
- Creating and managing sessions
- Session queries and filtering

Script Outline

[0:00-1:00] Introduction “Sessions are the foundation of organized AI-assisted development. Think of a session as a focused work period with a specific goal and mode.”

[1:00-3:00] Session Modes

```
// The 6 modes
session.ModePlanning      // Design and architecture
session.ModeBuilding      // Implementation
session.ModeTesting       // Quality assurance
session.ModeRefactoring   // Code improvement
session.ModeDebugging     // Issue investigation
session.ModeDeployment     // Release preparation
```

“Each mode optimizes the AI assistant’s behavior for that type of work.”

[3:00-5:00] Session Lifecycle

```
// Create
sess := mgr.Create("feature-name", session.ModeBuilding,
                  "project-id")

// Start (status: idle → active)
```

```
mgr.Start(sess.ID)

// Pause (status: active → paused)
mgr.Pause(sess.ID)

// Resume (status: paused → active)
mgr.Resume(sess.ID)

// Complete (status: active → completed)
mgr.Complete(sess.ID)

// Fail (status: active → failed)
mgr.Fail(sess.ID, "reason")
```

[5:00-7:00] Session Properties

```
// Rich metadata
sess.Name = "implement-payment-api"
sess.Mode = session.ModeBuilding
sess.Status = session.StatusActive
sess.ProjectID = "payment-service"
sess.AddTag("api")
sess.AddTag("payments")
sess.SetMetadata("sprint", "23")
sess.SetContext("last_file", "payment.go")
```

[7:00-9:00] Queries and Filtering

```
// Get by project
sessions := mgr.GetByProject("payment-service")

// Get by mode
buildingSessions := mgr.GetByMode(session.ModeBuilding)

// Get by status
activeSessions := mgr.GetByStatus(session.StatusActive)

// Get by tag
apiSessions := mgr.GetByTag("api")

// Search by name
sessions := mgr.FindByName("payment")

// Get recent
recent := mgr.GetRecent(10)
```

[9:00-10:00] Demo Live demonstration creating, starting, and querying sessions.

Key Takeaways

- Sessions organize work by mode and lifecycle
 - 6 modes cover all development activities
 - Rich metadata enables powerful filtering
 - Status transitions are tracked automatically
-

Video 4: Advanced Session Management (15 min)

Topics Covered

- Session history and statistics
- Export/import
- Callbacks and events
- Multi-session workflows
- Best practices

Script Outline

[0:00-3:00] Session History

```
// Get session statistics
stats := mgr.GetStatistics()
fmt.Printf("Total: %d\n", stats.TotalSessions)
fmt.Printf("Active: %d\n", stats.ActiveSessions)
fmt.Printf("By mode: %v\n", stats.ByMode)
fmt.Printf("By status: %v\n", stats.ByStatus)
```

```
// Trim old sessions (keep last 50)
mgr.TrimHistory(50)
```

```
// Count operations
total := mgr.Count()
byStatus := mgr.CountByStatus()
```

[3:00-6:00] Export and Import

```
// Export session for sharing
snapshot, err := mgr.Export(sess.ID)
data, _ := json.Marshal(snapshot)
os.WriteFile("session.json", data, 0644)
```

```
// Import session
var snapshot SessionSnapshot
json.Unmarshal(data, &snapshot)
mgr.Import(&snapshot)
```

“Export sessions to share with teammates or backup important work.”

[6:00-9:00] Callbacks and Events

```
// React to session events
mgr.OnCreate(func(s *Session) {
    fmt.Printf("Session created: %s\n", s.Name)
    // Log to analytics
})

mgr.OnStart(func(s *Session) {
    fmt.Printf("Session started: %s\n", s.Name)
    // Start timer, notify team
})

mgr.OnComplete(func(s *Session) {
    fmt.Printf("Session completed: %s\n", s.Name)
    // Generate report, update project status
})
```

[9:00-12:00] Multi-Session Workflows

```
// Work on multiple features simultaneously
authSession := mgr.Create("auth", session.ModeBuilding, "api")
testSession := mgr.Create("tests", session.ModeTesting, "api")

mgr.Start(authSession.ID)
// ... work on auth
mgr.Pause(authSession.ID)

// Switch to testing
mgr.Start(testSession.ID)
// ... write tests
mgr.Complete(testSession.ID)

// Resume auth work
mgr.Resume(authSession.ID)
```

[12:00-15:00] Best Practices - One active session per focus area - Use descriptive names and tags - Complete or fail sessions (don't leave orphaned) - Regular history trimming - Export important sessions for documentation

Video 5: Memory System Basics (10 min)

Topics Covered

- Messages and roles
- Conversations
- Token tracking
- Basic operations

Script Outline

[0:00-2:00] Understanding Messages

```
// Three role types
userMsg := memory.NewUserMessage("Help me debug this")
assistantMsg := memory.NewAssistantMessage("Sure, let's look
    at...")
systemMsg := memory.NewSystemMessage("You are a debugging
    expert")

// Messages have metadata
userMsg.SetMetadata("file", "main.go")
userMsg.SetMetadata("line", "42")

// Token counting
tokens := userMsg.EstimateTokens() // Approximate count
```

[2:00-5:00] Working with Conversations

```
// Create conversation
conv := mgr.CreateConversation("Feature Implementation")
conv.SessionID = "session-123" // Link to session

// Add messages
mgr.AddMessage(conv.ID, systemMsg)
mgr.AddMessage(conv.ID, userMsg)
mgr.AddMessage(conv.ID, assistantMsg)

// Retrieve messages
all := conv.GetMessages()
userOnly := conv.GetByRole(memory.RoleUser)
recent := conv.GetRecent(5)
```

[5:00-7:00] Searching and Filtering

```
// Search message content
results := conv.Search("debug")

// Get message range
messages := conv.GetRange(10, 20)

// Convert to text
text := conv.ToText() // Formatted conversation
```

[7:00-10:00] Token Management

```
// Check token counts
total := conv.TotalTokens()
fmt.Printf("Conversation uses %d tokens\n", total)

// Set limits
conv.SetMaxMessages(100)
conv.SetMaxTokens(10000)

// Automatic trimming when limits exceeded
mgr.AddMessage(conv.ID, newMsg)
// Oldest messages removed if over limit
```

Key Takeaways

- Three message roles: user, assistant, system
 - Conversations group related messages
 - Token tracking prevents API limits
 - Automatic trimming maintains bounds
-

Video 6: Advanced Memory Features (15 min)

Topics Covered

- Conversation management
- Statistics and analytics
- Export/import
- Performance optimization

Script Outline

[0:00-4:00] Advanced Queries

```

// Get all conversations
all := mgr.GetAll()

// Filter by session
convs := mgr.GetBySession("session-123")

// Search across conversations
results := mgr.SearchMessages("authentication")

// Get recent conversations
recent := mgr.GetRecent(10)

// Statistics
totalMsgs := mgr.TotalMessages()
totalTokens := mgr.TotalTokens()

```

[4:00-7:00] Conversation Statistics

```

// Per-conversation stats
stats := conv.GetStatistics()
fmt.Printf("Messages: %d\n", stats.TotalMessages)
fmt.Printf("User messages: %d\n", stats.UserMessages)
fmt.Printf("Assistant messages: %d\n", stats.AssistantMessages)
fmt.Printf("Tokens: %d\n", stats.TotalTokens)
fmt.Printf("Duration: %v\n", stats.Duration)

// Manager-wide stats
mgrStats := mgr.GetStatistics()
fmt.Printf("Total conversations: %d\n",
    mgrStats.TotalConversations)
fmt.Printf("Total messages: %d\n", mgrStats.TotalMessages)

```

[7:00-10:00] Export and Import

```

// Export conversation
snapshot := mgr.ExportConversation(conv.ID)
data, _ := json.MarshalIndent(snapshot, "", " ")
os.WriteFile("conversation.json", data, 0644)

// Import conversation
var snapshot memory.ConversationSnapshot
json.Unmarshal(data, &snapshot)
mgr.ImportConversation(&snapshot)

// Bulk export

```

```

for _, conv := range mgr.GetAll() {
    snapshot := mgr.ExportConversation(conv.ID)
    // Save to archive
}

```

[10:00-13:00] Performance Optimization

```

// Set appropriate limits
conv.SetMaxMessages(500)           // Reasonable conversation size
conv.SetMaxTokens(50000)           // Below API limits

// Trim old conversations
mgr.TrimConversations(50)           // Keep last 50

// Clear completed conversations
for _, conv := range mgr.GetAll() {
    if conv.SessionID != "" {
        sess, _ := sessionMgr.Get(conv.SessionID)
        if sess.Status == session.StatusCompleted {
            mgr.ClearConversation(conv.ID)
        }
    }
}

// Delete old conversations
old := time.Now().AddDate(0, -3, 0) // 3 months ago
for _, conv := range mgr.GetAll() {
    if conv.CreatedAt.Before(old) {
        mgr.DeleteConversation(conv.ID)
    }
}

```

[13:00-15:00] Best Practices - Set limits early to prevent unbounded growth - Regularly export important conversations - Trim or delete old conversations - Use metadata for organization - Monitor token usage

Video 7: State Persistence Fundamentals (10 min)

Topics Covered

- Why persistence matters
- Storage formats
- Save and load operations
- Data integrity

Script Outline

[0:00-2:00] Why Persistence? “Imagine spending hours building perfect context with your AI assistant, then losing it all to a crash. State persistence prevents this - nothing is ever lost.”

[2:00-5:00] Storage Formats

```
// JSON - Human readable, easy to inspect
store := persistence.NewStore("./data")
store.SetFormat(persistence.FormatJSON)

// Compact JSON - Smaller files
store.SetFormat(persistence.FormatCompactJSON)

// JSON + GZIP - Compressed, smallest size
store.SetFormat(persistence.FormatJSONGZIP)

// Compare sizes
// JSON:          150 KB
// Compact:       120 KB
// GZIP:          40 KB
```

[5:00-7:00] Basic Operations

```
// Save all state
err := store.Save()

// Save specific components
store.SaveSessions()
store.SaveConversations()
store.SaveTemplates()

// Load all state
err := store.Load()

// Load specific components
store.LoadSessions()
store.LoadConversations()
store.LoadTemplates()
```

[7:00-10:00] Data Integrity

```
// Atomic writes prevent corruption
store.Save() // Writes to temp file, then renames
```

```
// Validation on load
if err := store.Load(); err != nil {
    // Corrupted or missing data
    fmt.Printf("Load failed: %v\n", err)
}

// Get last save time
lastSave := store.LastSaveTime()
fmt.Printf("Last saved: %v\n", lastSave)

// Check file existence
if _, err := os.Stat("./data/sessions.json"); err == nil {
    // File exists
}
```

Key Takeaways

- Persistence prevents data loss
 - Three formats: JSON, Compact JSON, GZIP
 - Atomic writes ensure integrity
 - Validation detects corruption
-

Video 8: Advanced Persistence (10 min)

Topics Covered

- Auto-save configuration
- Backup and restore
- Migration strategies
- Error handling

Script Outline

[0:00-3:00] Auto-Save

```
// Enable auto-save (interval in seconds)
store.EnableAutoSave(300) // Every 5 minutes

// Disable auto-save
store.DisableAutoSave()

// Check if enabled
if store.IsAutoSaveEnabled() {
```

```

    interval := store.GetAutoSaveInterval()
    fmt.Printf("Auto-save every %d seconds\n", interval)
}

```

```

// Manual save during auto-save
store.Save() // Safe to call anytime

```

[3:00-6:00] Backup and Restore

```

// Create backup
backupPath := "./backups/backup-" +
    time.Now().Format("20060102-150405")
err := store.Backup(backupPath)

// List backups
backups, _ := filepath.Glob("./backups/backup-*")
for _, b := range backups {
    info, _ := os.Stat(b)
    fmt.Printf("%s (%d KB)\n", b, info.Size()/1024)
}

// Restore from backup
err := store.Restore("./backups/backup-20250107-120000")

// Restore with validation
if err := store.Restore(backupPath); err != nil {
    fmt.Printf("Restore failed: %v\n", err)
    // Fallback to earlier backup
}

```

[6:00-8:00] Migration

```

// Detect format
format := persistence.DetectFormat("./data/sessions.json")

// Convert formats
oldStore := persistence.NewStore("./old-data")
oldStore.SetFormat(persistence.FormatJSON)

newStore := persistence.NewStore("./new-data")
newStore.SetFormat(persistence.FormatJSONGZIP)

// Load from old
oldStore.Load()

```

```
// Save to new
newStore.SetSessionManager(sessionMgr)
newStore.Save()
```

[8:00-10:00] Error Handling

```
// Callbacks for events
store.OnSave(func() {
    fmt.Println("State saved successfully")
})

store.OnLoad(func() {
    fmt.Println("State loaded successfully")
})

store.OnError(func(err error) {
    fmt.Printf("Persistence error: %v\n", err)
    // Log, alert, retry, etc.
})

// Graceful degradation
if err := store.Load(); err != nil {
    // Start fresh but log the issue
    log.Printf("Could not load state: %v\n", err)
    // Continue with empty state
}
```

Key Takeaways

- Auto-save eliminates manual saves
 - Regular backups protect against corruption
 - Format migration is straightforward
 - Error callbacks enable monitoring
-

Video 9: Template System Basics (10 min)

Topics Covered

- Template types
- Variable substitution
- Creating templates
- Built-in templates

Script Outline

[0:00-2:00] Template Types

```
template.TypeCode           // Code generation
template.TypePrompt        // AI prompts
template.TypeWorkflow       // Process templates
template.TypeDocumentation  // Documentation
template.TypeEmail          // Email templates
template.TypeCustom         // Custom use
```

[2:00-4:00] Variable Substitution

```
// Template content with placeholders
content := `func {{function_name}}({{parameters}})
    {{return_type}} {
    {{body}}
}`

// Rendering replaces {{placeholders}}
result, _ := tpl.Render(map[string]interface{}{
    "function_name": "ProcessData",
    "parameters":    "data []byte",
    "return_type":   "error",
    "body":          "return nil",
})

// Result:
// func ProcessData(data []byte) error {
//     return nil
// }
```

[4:00-6:00] Creating Templates

```
// Create template
tpl := template.NewTemplate(
    "API Handler",
    "Generate HTTP handler",
    template.TypeCode,
)

// Set content
tpl.SetContent(`func Handle{{name}}(w http.ResponseWriter, r
    *http.Request) {
    // {{description}}
    {{implementation}}
}
```

```

})

// Add variables
tpl.AddVariable(template.Variable{
    Name:      "name",
    Required:   true,
    Type:      "string",
})

tpl.AddVariable(template.Variable{
    Name:      "description",
    Required:   false,
    DefaultValue: "TODO: Add description",
    Type:      "string",
})

tpl.AddVariable(template.Variable{
    Name:      "implementation",
    Required:   true,
    Type:      "string",
})

// Add metadata
tpl.AddTag("http")
tpl.AddTag("handler")
tpl.SetMetadata("language", "go")

```

[6:00-10:00] Built-in Templates

```

// Load built-in templates
mgr := template.NewManager()
mgr.RegisterBuiltinTemplates()

// 1. Function template
result, _ := mgr.RenderByName("Function", map[string]interface{}{
    "function_name": "calculate",
    "parameters":    "a, b int",
    "return_type":   "int",
    "body":          "return a + b",
})

// 2. Code Review template
result, _ := mgr.RenderByName("Code Review",
    map[string]interface{}{
        "language": "Go",
    })

```

```

        "code":      sourceCode,
    })

// 3. Bug Fix template
result, _ := mgr.RenderByName("Bug Fix", map[string]interface{}{
    "language":      "Go",
    "error_message": "nil pointer dereference",
    "code":          buggyCode,
    "expected_behavior": "Should handle nil values",
    "actual_behavior":  "Crashes on nil",
})

// 4. Function Documentation
result, _ := mgr.RenderByName("Function Documentation",
    map[string]interface{}{
        "function_name": "ProcessData",
        "description":   "processes incoming data",
    })

// 5. Status Update Email
result, _ := mgr.RenderByName("Status Update Email",
    map[string]interface{}{
        "project_name":      "API Server",
        "recipient_name":    "Team",
        "progress":          75,
        "status":            "On Track",
        "completed_items":   "- Auth\n- Database",
        "in_progress_items": "- Testing",
        "next_steps":        "- Deploy",
        "sender_name":       "Development Team",
    })

```

Key Takeaways

- 6 template types for different uses
 - {{variable}} syntax for substitution
 - Required vs optional variables
 - 5 production-ready built-in templates
-

Video 10: Advanced Templates (15 min)

Topics Covered

- Template manager features
- Search and filtering
- File operations
- Export/import
- Template libraries

Script Outline

[0:00-3:00] Template Manager

```
// Register custom template
mgr := template.NewManager()
mgr.Register(myTemplate)

// Get templates
tpl, _ := mgr.Get("template-id")
tpl, _ := mgr.GetByName("API Handler")

// Get by type
codeTpls := mgr.GetByType(template.TypeCode)
promptTpls := mgr.GetByType(template.TypePrompt)

// Get by tag
httpTpls := mgr.GetByTag("http")

// Search
results := mgr.Search("handler") // Searches name, description,
                                tags
```

[3:00-6:00] File Operations

```
// Save template to file
mgr.SaveToFile("tpl-123", "./templates/api-handler.json")

// Load from file
mgr.LoadFromFile("./templates/api-handler.json")

// Load entire directory
count, _ := mgr.LoadFromDirectory("./templates")
fmt.Printf("Loaded %d templates\n", count)
```



```
// Export all templates
for _, tpl := range mgr.GetAll() {
    filename := fmt.Sprintf("./export/%s.json", tpl.Name)
    mgr.SaveToFile(tpl.ID, filename)
}
```

[6:00-9:00] Template Export/Import

```
// Export for sharing
snapshot, _ := mgr.Export("tpl-123")
data, _ := json.MarshalIndent(snapshot, "", " ")
os.WriteFile("template.json", data, 0644)
```

```
// Import shared template
var snapshot template.TemplateSnapshot
json.Unmarshal(data, &snapshot)
mgr.Import(&snapshot)
```

```
// Share with team
// Templates are portable JSON files
```

[9:00-12:00] Building Template Libraries

```
// Organize by project/domain
projectTemplates := mgr.GetByTag("ecommerce")
apiTemplates := mgr.GetByTag("api")
```

```
// Template sets for workflows
workflowTpls := []string{
    "Feature Planning",
    "Implementation",
    "Testing",
    "Documentation",
    "Deployment",
}
```

```
for _, name := range workflowTpls {
    tpl, _ := mgr.GetByName(name)
    // Use in workflow
}
```

```
// Version templates
tpl.Version = "2.0.0"
tpl.SetMetadata("changelog", "Added error handling")
```

```
// Statistics
stats := mgr.GetStatistics()
fmt.Printf("Total templates: %d\n", stats.TotalTemplates)
fmt.Printf("By type: %v\n", stats.ByType)
fmt.Printf("Tag cloud: %v\n", stats.TagCloud)
```

[12:00-15:00] Advanced Patterns

```
// Callbacks for template events
mgr.OnCreate(func(t *template.Template) {
    fmt.Printf("New template: %s\n", t.Name)
})

mgr.OnUpdate(func(t *template.Template) {
    fmt.Printf("Updated: %s\n", t.Name)
})

// Dynamic templates
content := buildDynamicTemplate(spec)
tpl := template.ParseTemplate("Dynamic", content,
    template.TypeCode)

// Template composition
header := headerTemplate.Render(vars)
body := bodyTemplate.Render(vars)
footer := footerTemplate.Render(vars)
complete := header + body + footer

// Template validation
extracted := tpl.ExtractVariables()
for _, varName := range extracted {
    if !hasVariable(tpl.Variables, varName) {
        fmt.Printf("Warning: undeclared variable %s\n", varName)
    }
}
}
```

Key Takeaways

- Manager provides powerful template operations
 - File I/O enables template libraries
 - Export/import for sharing
 - Organize with tags and metadata
 - Callbacks for automation
-

Video 11: Integration Patterns (12 min)

Topics Covered

- Session + Memory integration
- Templates + Conversations
- Full workflow examples
- Best practices

Script Outline

[0:00-3:00] Session + Memory

```
// Create session and conversation together
sess := sessionMgr.Create("feature-impl", session.ModeBuilding,
    "api")
sessionMgr.Start(sess.ID)

conv := memoryMgr.CreateConversation("Implementation Discussion")
conv.SessionID = sess.ID // Link them

// Session context flows into messages
memoryMgr.AddMessage(conv.ID, memory.NewSystemMessage(
    fmt.Sprintf("Current session: %s, Mode: %s", sess.Name,
        sess.Mode),
    ))

// Complete session when done
sessionMgr.Complete(sess.ID)
conv.SetMetadata("session_status", "completed")
```

[3:00-6:00] Templates + Conversations

```
// Use template to generate prompt
prompt, _ := templateMgr.RenderByName("Code Review",
    map[string]interface{}{
        "language": "Go",
        "code":     sourceCode,
    })

// Add prompt to conversation
memoryMgr.AddMessage(conv.ID, memory.NewUserMessage(prompt))

// Simulate AI response (in real use, send to LLM)
response := callLLM(prompt)
```

```
memoryMgr.AddMessage(conv.ID,
    memory.NewAssistantMessage(response))

// Generate code from template
code, _ := templateMgr.RenderByName("Function", vars)
memoryMgr.AddMessage(conv.ID, memory.NewAssistantMessage(
    fmt.Sprintf("Here's the implementation:\n``go\n%s\n``",
        code),
    ))
```

[6:00-9:00] Complete Workflow

```
func implementFeature(featureName string) {
    // 1. Planning session
    planSess := sessionMgr.Create(featureName+"-plan",
        session.ModePlanning, "api")
    sessionMgr.Start(planSess.ID)

    planConv := memoryMgr.CreateConversation("Planning: " +
        featureName)
    planConv.SessionID = planSess.ID

    // Discuss with AI, gather requirements
    // ...

    sessionMgr.Complete(planSess.ID)

    // 2. Implementation session
    buildSess := sessionMgr.Create(featureName,
        session.ModeBuilding, "api")
    sessionMgr.Start(buildSess.ID)

    buildConv := memoryMgr.CreateConversation("Implementation: "
        + featureName)
    buildConv.SessionID = buildSess.ID

    // Generate code using templates
    code, _ := templateMgr.RenderByName("Function", vars)
    memoryMgr.AddMessage(buildConv.ID,
        memory.NewAssistantMessage(code))

    // 3. Testing session
    testSess := sessionMgr.Create(featureName+"-test",
        session.ModeTesting, "api")
    sessionMgr.Start(testSess.ID)
```

```

testConv := memoryMgr.CreateConversation("Testing: " +
    featureName)
testConv.SessionID = testSess.ID

// Generate tests using templates
tests, _ := templateMgr.RenderByName("Test Suite", vars)

// All saved automatically via persistence
sessionMgr.Complete(testSess.ID)
sessionMgr.Complete(buildSess.ID)
}

```

[9:00-12:00] Best Practices

```

// 1. One conversation per session
sess := sessionMgr.Create("feature", session.ModeBuilding,
    "project")
conv := memoryMgr.CreateConversation(sess.Name)
conv.SessionID = sess.ID

// 2. Use templates for consistency
prompt, _ := templateMgr.RenderByName("Code Review", vars)
// Don't hardcode prompts repeatedly

// 3. Tag everything for searchability
sess.AddTag("api")
sess.AddTag("authentication")
conv.SetMetadata("feature", "auth")

// 4. Export important work
sessionMgr.Export(sess.ID)
memoryMgr.ExportConversation(conv.ID)

// 5. Clean up completed work
if sess.Status == session.StatusCompleted {
    // Export first, then trim
    sessionMgr.Export(sess.ID)
    // Archive conversation
}

```

Key Takeaways

- Link sessions and conversations via SessionID
- Templates generate content for conversations
- Multi-session workflows for complex features

- Tag and organize for discoverability
 - Export important work before cleanup
-

Video 12: Advanced Workflows (13 min)

Topics Covered

- Multi-session development
- Code review workflow
- Debugging sessions
- Template-based generation
- Performance optimization

Script Outline

[0:00-3:00] Multi-Session Development

```
// Working on multiple features in parallel
func parallelDevelopment() {
    // Feature 1: Authentication
    authSess := sessionMgr.Create("auth", session.ModeBuilding,
        "api")
    sessionMgr.Start(authSess.ID)
    authConv := memoryMgr.CreateConversation("Auth
        Implementation")
    authConv.SessionID = authSess.ID

    // Do some auth work
    // ...
    sessionMgr.Pause(authSess.ID) // Pause when switching

    // Feature 2: Payments
    paySess := sessionMgr.Create("payments",
        session.ModeBuilding, "api")
    sessionMgr.Start(paySess.ID)
    payConv := memoryMgr.CreateConversation("Payment
        Integration")
    payConv.SessionID = paySess.ID

    // Work on payments
    // ...
    sessionMgr.Pause(paySess.ID)

    // Resume auth
```

```

    sessionMgr.Resume(authSess.ID)
    // Context is preserved
}

```

[3:00-6:00] Code Review Workflow

```

func codeReviewWorkflow(pr *PullRequest) {
    // Create review session
    sess := sessionMgr.Create(
        fmt.Sprintf("review-pr-%d", pr.Number),
        session.ModeBuilding, // Or create ModeReviewing
        pr.Project,
    )
    sess.AddTag("review")
    sess.AddTag(fmt.Sprintf("pr-%d", pr.Number))
    sessionMgr.Start(sess.ID)

    conv := memoryMgr.CreateConversation(pr.Title)
    conv.SessionID = sess.ID

    // Review each file
    for _, file := range pr.Files {
        // Use code review template
        prompt, _ := templateMgr.RenderByName("Code Review",
            map[string]interface{}{
                "language":    detectLanguage(file.Name),
                "code":          file.Content,
                "focus_areas": "security, performance, best
practices",
            })

        memoryMgr.AddMessage(conv.ID,
            memory.NewUserMessage(prompt))

        // Get AI feedback
        feedback := callLLM(prompt)
        memoryMgr.AddMessage(conv.ID,
            memory.NewAssistantMessage(feedback))

        // Store feedback
        pr.AddComment(file.Name, feedback)
    }

    // Generate review summary
    summaryPrompt := "Summarize the code review findings"

```

```

memoryMgr.AddMessage(conv.ID,
    memory.NewUserMessage(summaryPrompt))

summary := callLLM(conv.ToText() + "\n" + summaryPrompt)

sessionMgr.Complete(sess.ID)
sess.SetMetadata("review_summary", summary)
}

```

[6:00-9:00] Debugging Workflow

```

func debuggingWorkflow(issue *Issue) {
    // Debug session
    sess := sessionMgr.Create(
        fmt.Sprintf("debug-%s", issue.ID),
        session.ModeDebugging,
        issue.Project,
    )
    sess.SetMetadata("issue_id", issue.ID)
    sess.SetMetadata("severity", issue.Severity)
    sessionMgr.Start(sess.ID)

    conv := memoryMgr.CreateConversation(issue.Title)
    conv.SessionID = sess.ID

    // Initial context
    memoryMgr.AddMessage(conv.ID, memory.NewSystemMessage(
        "You are debugging a production issue.",
    ))

    // Use bug fix template
    prompt, _ := templateMgr.RenderByName("Bug Fix",
        map[string]interface{}{
            "language":      "Go",
            "error_message":  issue.Error,
            "code":          issue.StackTrace,
            "expected_behavior": issue.Expected,
            "actual_behavior": issue.Actual,
        })

    memoryMgr.AddMessage(conv.ID, memory.NewUserMessage(prompt))

    // Iterative debugging
    for !issue.Resolved {
        response := callLLM(conv.ToText())
    }
}

```



```

memoryMgr.AddMessage(conv.ID,
memory.NewAssistantMessage(response))

// Try suggested fix
testResult := testFix(response)
memoryMgr.AddMessage(conv.ID, memory.NewUserMessage(
    fmt.Sprintf("Test result: %s", testResult),
))

if testResult == "success" {
    issue.Resolved = true
    sessionMgr.Complete(sess.ID)
}

// Export for documentation
snapshot, _ := sessionMgr.Export(sess.ID)
// Save to issue tracker
}

```

[9:00-11:00] Template-Based Generation

```

// Generate entire features from templates
func generateCRUD(entity string) {
    // Model template
    model, _ := templateMgr.RenderByName("Model",
        map[string]interface{}{
            "name": entity,
            "fields": getEntityFields(entity),
        })

    // Repository template
    repo, _ := templateMgr.RenderByName("Repository",
        map[string]interface{}{
            "entity": entity,
        })

    // Service template
    service, _ := templateMgr.RenderByName("Service",
        map[string]interface{}{
            "entity": entity,
        })

    // Handler template

```

```

handler, _ := templateMgr.RenderByName("HTTP Handler",
    map[string]interface{}{
        "entity": entity,
        "methods": []string{"GET", "POST", "PUT", "DELETE"},
    })

// Tests template
tests, _ := templateMgr.RenderByName("Test Suite",
    map[string]interface{}{
        "entity": entity,
    })

// Write files
writeFile(fmt.Sprintf("models/%s.go", entity), model)
writeFile(fmt.Sprintf("repositories/%s.go", entity), repo)
writeFile(fmt.Sprintf("services/%s.go", entity), service)
writeFile(fmt.Sprintf("handlers/%s.go", entity), handler)
writeFile(fmt.Sprintf("tests/%s_test.go", entity), tests)
}

```

[11:00-13:00] Performance Tips

```

// 1. Set appropriate limits
conv.SetMaxMessages(500)
conv.SetMaxTokens(50000)

// 2. Trim regularly
sessionMgr.TrimHistory(50)
memoryMgr.TrimConversations(50)

// 3. Use GZIP for large datasets
store.SetFormat(persistence.FormatJSONGZIP)

// 4. Auto-save intervals
store.EnableAutoSave(300) // 5 minutes, not too frequent

// 5. Lazy loading
// Don't load all conversations at startup
// Load on demand:
conv, _ := memoryMgr.Get(convID)

// 6. Batch operations
for _, tpl := range templates {
    mgr.Register(tpl) // Registration is fast
}

```

```
// Single save at the end
store.Save()

// 7. Use callbacks for monitoring
sessionMgr.OnCreate(logEvent)
sessionMgr.OnComplete(updateMetrics)

// 8. Archive old data
if sess.CompletedAt.Before(time.Now().AddDate(0, -6, 0)) {
    // Export and delete
    sessionMgr.Export(sess.ID)
    sessionMgr.Delete(sess.ID)
}
```

Key Takeaways

- Multi-session workflows handle complex projects
 - Specialized workflows for review, debugging, generation
 - Templates enable rapid scaffolding
 - Performance optimization maintains speed
 - Regular maintenance prevents bloat
-

Course Conclusion

Final Summary (2 min)

SCRIPT: “Congratulations on completing the Phase 3 course! You’ve mastered:

- Session Management for organized development
- Memory System for conversation context
- State Persistence for reliability
- Template System for productivity
- Integration patterns for real-world workflows

You now have the tools to build powerful, persistent AI-assisted development workflows. Use these features to enhance your productivity, maintain context across long projects, and create reusable knowledge.

Remember the key principles: - Organize work with sessions - Maintain context with memory - Never lose progress with persistence - Accelerate with templates - Integrate all systems for maximum benefit

Happy coding with HelixCode Phase 3!”

Certification

- Complete all 12 videos
- Pass module quizzes (70% minimum)
- Complete final project
- Receive HelixCode Phase 3 Certified Developer badge

Resources

- Documentation: /docs
 - Examples: /examples
 - Community: GitHub Discussions
 - Support: GitHub Issues
-

End of Video Scripts